# 🛡 ShopShield

Website Risk & Online Shopping Scam Intelligence System

VERSION 1.0.0
OCTOBER 2026

## 1. Executive Summary & Transformation

The original ShopShield repository started as a single static HTML demonstration ( `shopshield.html` ) using rudimentary string-based heuristic points (e.g. basic keyword presence, URL length, hardcoded scores).

The project has undergone a **complete architectural and functional rebuild** into an authentic, production-grade **Full-Stack Risk Intelligence Platform**. The final system executes real multi-layered server-side investigations—probing DNS infrastructure, validating SSL/TLS cryptographic certificates, tracing redirect hops, querying ICANN registration databases (RDAP), statically parsing HTML content for consumer policies, and identifying manipulative e-commerce tactics.

> **Core Philosophy:** Real Functionality > Fake AI. Evidence > Arbitrary Scores. Transparent Risk Intelligence > Unverifiable Safe Verdicts.

## 2. Implemented Real-World Functionality

### A. Strict SSRF Defense & URL Normalization

- **Protocol Validation:** Accepts standard web addresses (e.g., `example.com` , `https://...` , `http://...` ), parses punycode/IDN homographs ( `xn--...` ), and strips misleading userinfo (e.g., `brand.com@evil.com` ).
- **Dual-Stage SSRF Filter:** Automatically inspects resolved hostnames and blocks private, loopback, link-local, carrier-grade NAT, and cloud metadata IP ranges (RFC 1918 `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` ; Loopback `127.0.0.0/8` , `::1` ; AWS/GCP metadata `169.254.169.254` ; Multicast `224.0.0.0/4` ).

### B. Safe Network Fetcher & Redirect Tracer

- **Controlled Requests:** Requests operate under strict 7-second timeouts with a 2MB maximum response payload cap to prevent Denial-of-Service and memory exhaustion.
- **Per-Hop SSRF Verification:** Follows up to 5 HTTP redirects manually, verifying each intermediate target host against SSRF rules.
- **Cross-Domain Switch Detection:** Flags instances where a user-submitted address silently redirects to a completely different domain (a hallmark of deceptive pop-up storefronts).

### C. DNS & Cryptographic TLS Inspection

- **DNS Records:** Resolves IPv4 (A), IPv6 (AAAA), Mail Exchange (MX), and Text (TXT) records. Detects missing MX mail servers, common on disposable scam websites.
- **TLS Socket Handshake:** Establishes direct TLS socket connections to inspect peer certificates, CA issuers (e.g., DigiCert, Let's Encrypt), validity dates, days remaining until expiration, and Subject Alternative Name (SAN) hostname matching.

### D. ICANN RDAP Domain Age & Registrar Intelligence

- **Direct Registry Queries:** Queries official IETF RFC 7482 standardized RDAP endpoints to obtain verifiable creation dates and registrar credentials.
- **Domain Age Classification:** Identifies brand-new domains (< 30 days or < 90 days). If RDAP data is protected or unavailable, it is marked as `UNAVAILABLE` and **never penalized**.

### E. Static HTML Content & Policy Auditing

- **Zero Server-Side JS Execution:** Utilizes `cheerio` for memory-safe static DOM analysis without executing potentially malicious remote scripts.
- **Consumer Trust Policies:** Detects dedicated links to Refund/Return policies, Privacy policies, Terms of Service, Shipping guidelines, and "About Us" company backgrounds.
- **Contact Channels:** Identifies direct customer support emails ( `mailto:` ), telephone lines ( `tel:` ), and physical street addresses (including Schema.org `PostalAddress` JSON-LD).

### F. Online Shopping Threat & Pressure Detection

- **Untraceable Payment Coercion:** Flags requests demanding wire transfers, Western Union, cryptocurrency-only, or gift cards (Amazon, Apple, Steam).
- **Unrealistic Discount Lures:** Identifies extreme claims (e.g., "90% off everything today only", "warehouse blowout").
- **Artificial Urgency:** Detects countdown timers and artificial scarcity triggers ("Only 1 left in stock", "Cart expires in 04:59").

## 3. Risk Classifications & Evidence Synthesis

ShopShield discards arbitrary point arithmetic in favor of an evidence-first synthesis engine that separates **Risk Level** from **Data Confidence**.

| Risk Level | Criteria | Actionable Guidance |
|---|---|---|
| LOW RISK | Standard security baselines verified (HTTPS active, active MX records, clear return policy, verified contact info, standard payments). | No major red flags detected. Maintain normal vigilance when purchasing. |
| MODERATE RISK | Missing consumer baseline (e.g., newly registered domain, absent refund policy, or aggressive promotions). | Verify merchant contact details and reviews before submitting payment. |
| HIGH RISK | Multiple compounding indicators (e.g., domain < 30 days old + cross-domain redirect + extreme 90% discount claims + no contact info). | Avoid transacting. Exercise extreme caution; seek independent third-party reviews. |
| CRITICAL WARNING | Active threat intelligence match (Google Safe Browsing / VirusTotal) or untraceable payment coercion (wire/crypto/gift card). | Do NOT submit payment or credentials. Block unauthorized transactions immediately. |
| UNABLE TO ASSESS | Server unreachable, connection refused, DNS lookup failed, or target address blocked by SSRF filter. | Target site could not be inspected. Do not proceed on unverified hosts. |

## 4. Technology Stack & Project Structure

| Component | Technology | Purpose |
|---|---|---|
| **Full-Stack Framework** | Next.js 14 (App Router, Node.js Runtime) | Unified web UI, server actions, and API route handlers. |
| **Language** | TypeScript (Strict Mode) | End-to-end type safety for network protocols and risk data. |
| **DOM / HTML Parser** | Cheerio 1.0 | Fast, memory-safe static HTML parsing without JS execution. |
| **Domain Extraction** | tldts | Public Suffix List compliant domain and subdomain isolation. |
| **Validation** | Zod | Strict runtime validation of API requests and JSON payloads. |
| **Test Suite** | Vitest | High-speed unit and integration test runner (50/50 passing). |

```
shopshield/
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── analyze/route.ts       # POST /api/analyze (Main Scanner)
│   │   │   └── health/route.ts        # GET /api/health (Service Status)
│   │   ├── globals.css                # Polished responsive design system
│   │   ├── layout.tsx                 # Root layout & document metadata
│   │   └── page.tsx                   # Interactive dashboard & scanner UI
│   ├── lib/
│   │   ├── engine/riskEngine.ts       # Evidence synthesis & risk engine
│   │   ├── fetcher/safeFetch.ts       # SSRF-checked HTTP fetcher
│   │   ├── normalizer/urlNormalizer.ts# URL normalizer & punycode decoder
│   │   ├── pipeline.ts                # Concurrency orchestration pipeline
│   │   ├── security/ssrf.ts           # IP range filters & SSRF mitigation
│   │   └── services/
│   │       ├── contentAnalyzer.ts     # Cheerio HTML parser
│   │       ├── dnsService.ts          # Node DNS records resolver
│   │       ├── rdapService.ts         # ICANN RDAP domain history
│   │       ├── reputationService.ts   # Safe Browsing & VirusTotal
│   │       └── tlsService.ts          # Node TLS socket cert inspector
│   └── types/analyzer.ts              # Strongly-typed data models
├── tests/                             # 50 Automated unit & integration tests
├── .env.example                       # Documented optional API keys
├── next.config.mjs                    # Next.js configuration & legacy redirects
└── package.json                       # Dependencies and lifecycle scripts
```

## 5. Step-by-Step Guide: Running on Another Laptop

The repository is public on GitHub at:
**https://github.com/mayanksupadhye-source/shopshield**

Follow these exact steps to clone, configure, test, and run ShopShield on any computer (Windows, macOS, or Linux):

### 1 Verify Prerequisites

Ensure you have **Node.js 18+** (Node 20 or 24 LTS recommended) and **Git** installed. Check in your terminal:

```
node -v
npm -v
git --version
```

### 2 Clone the Public Repository

Open Terminal (macOS/Linux) or PowerShell (Windows) and clone the repository:

```
git clone https://github.com/mayanksupadhye-source/shopshield.git
cd shopshield
```

### 3 Install Project Dependencies

Install the required packages:

```
npm install
```

### 4 Environment Configuration (Optional)

ShopShield works out of the box with real DNS, TLS, RDAP, and content scanning without third-party API keys. If you want Google Safe Browsing or VirusTotal integration, copy the environment template:

```
# On Windows (PowerShell):
Copy-Item .env.example .env.local

# On macOS / Linux:
cp .env.example .env.local
```

### 5 Run Automated Tests

Run the 50 automated tests to verify network filters, SSRF defenses, and scoring rules on the new machine:

```
npm test
```

### 6 Start the Development Server

Launch the Next.js development server:

```
npm run dev
```

Open your web browser and navigate to:
👉 **http://localhost:3000**

### 7 Production Deployment (Optional)

To compile an optimized production build and run it in standalone production mode:

```
npm run build
npm start
```

## 6. Important Troubleshooting & Best Practices

- **Port 3000 in use:** If another service is running on port 3000, specify a different port: `PORT=3001 npm run dev`.
- **Do NOT open `shopshield.html`:** The old static file was replaced with the Next.js application. Always open `http://localhost:3000` in your browser.
- **Honest Security Limitations:** A site with valid HTTPS, a modern design, and standard policies is not guaranteed to be legitimate. Conversely, a newly registered shop is not guaranteed to be fraudulent. ShopShield provides verifiable risk intelligence to help consumers make informed purchasing decisions.